SimpleSmartLoader

Operating System : Assignment 4

Group Number: 161

Group Member: Mihir Jaiswal

Implementation

ELF File Reading and Validation

The loader receives the executable path from the launcher and opens the ELF file in read-only mode. It reads the file contents into memory and validates the ELF magic, 32-bit class, little-endian format, Intel 80386 machine type, executable file type, and program-header table bounds. The program-header table is then used to identify the loadable segments.

Lazy Loading

Unlike a loader that maps every loadable segment before execution, SimpleSmartLoader does not allocate PT_LOAD segments upfront. It checks that the ELF entry point lies inside an executable PT_LOAD segment, installs a SIGSEGV handler, and calls the entry point by converting its address to a function pointer. The first access to an unmapped segment page causes a segmentation fault.

Page-Fault Handling and Page Allocation

The SIGSEGV handler obtains the faulting address and checks whether it belongs to a PT_LOAD segment. For a valid missing page, it aligns the address down to a 4096-byte boundary and invokes the page-loading routine. The routine finds all loadable segments overlapping that page, determines the required memory permissions, and maps one page using mmap(). It then copies the file-backed bytes for that page. Anonymous mmap memory is initially zero-filled, so bytes belonging to the BSS portion remain zero. Finally, mprotect() applies the segment permissions and execution resumes after the fault is handled.

Page Tracking and Statistics

The loader records the virtual address of each allocated page to avoid allocating the same page again. It maintains counters for page faults and page allocations. Internal fragmentation is calculated by subtracting the number of bytes occupied by loadable segment memory in each allocated page from 4096 bytes, and accumulating the unused bytes. At the end of execution, the loader prints the user _start return value, total page faults, total page allocations, and internal fragmentation in KB.

Cleanup and Error Handling

The loader checks important operations such as open(), fstat(), read(), malloc(), calloc(), sigaction(), mmap(), and mprotect(). It validates ELF metadata and segment bounds before execution. The cleanup routine restores the previous SIGSEGV action, unmaps allocated pages, frees bookkeeping and ELF-file memory, and closes the file descriptor.

Testing

SUM Test

The SUM test executable was built as a 32-bit ELF executable and run through the launcher. The observed return value was 2048. The recorded output showed 3 page faults, 3 page allocations, and 7.889 KB of internal fragmentation.

FIB Test

The FIB test executable was built as a 32-bit ELF executable and run through the launcher. The observed return value was 102334155. The recorded output showed 1 page fault, 1 page allocation, and 3.889 KB of internal fragmentation.

Clean Build

The project was cleaned and rebuilt using make clean followed by make. The test programs, shared loader library, and launcher were rebuilt successfully without compiler errors.

Limitations and Points to Verify

Execution Environment

The implementation targets 32-bit little-endian Intel ELF executable files of type ET_EXEC and is intended for the assignment test programs, which do not use glibc APIs.

Signal Handler Safety

The current implementation performs page mapping, memory copying, and protection changes from the SIGSEGV handler. These operations are not generally guaranteed to be async-signal-safe, so this design choice should be reviewed against the assignment expectations.

Additional Testing

The SUM and FIB tests pass, but additional invalid-input and memory-protection tests should be run before submission. In particular, invalid fault addresses, overlapping segment pages, and read/write/execute permissions should be checked.

Conclusion

SimpleSmartLoader demonstrates demand paging for 32-bit ELF executables. It avoids upfront mapping of PT_LOAD segments, handles page faults by allocating individual 4 KB pages, copies the required segment contents, applies memory permissions, and reports page-fault, page-allocation, and internal-fragmentation statistics. The clean build and the SUM and FIB test results provide initial evidence that the implementation works for the supplied test programs.

Group Member Contributions

Mihir Jaiswal: implementation of the ELF loader, lazy page allocation, page-fault handling, statistics, build configuration, and testing. If this is a group submission, add the other member's name and their actual contribution before submitting.

GitHub Repository

https://github.com/Mihir2024347/SimpleSmartLoader.git